

Algorithms

Assignment #3

Yoon Suk Kang

School of Computer Science
Chungbuk National University



충북대학교
CHUNGBUK NATIONAL UNIVERSITY
서원구 충대로 1

Assignment #4: Graph Algorithms

□ Overview: implementing two graph algorithms

- (1) Single-Source Shortest Paths: **Dijkstra's algorithm**
- (2) All-pairs shortest paths: **Floyd-Warshall algorithm**

Algorithm to implement (1/2)

□ Dijkstra's Algorithm

■ Any data structure can be used

- Array
- Binary heap
- ...

DIJKSTRA(G, w, s)

INIT-SINGLE-SOURCE(G, s)

$S = \emptyset$

$Q = G.V$

// Priority queue is sorted in the increasing order by the distance from the source

while $Q \neq \emptyset$

$u = \text{EXTRACT-MIN}(Q)$

// the closest vertex to the source is picked

$S = S \cup \{u\}$

for each vertex $v \in G.Adj[u]$

RELAX(u, v, w)

Algorithm to implement (2/2)

□ Floyd-Warshall Algorithm

■ Adjacency-matrix representation

FLOYD-WARSHALL(W)

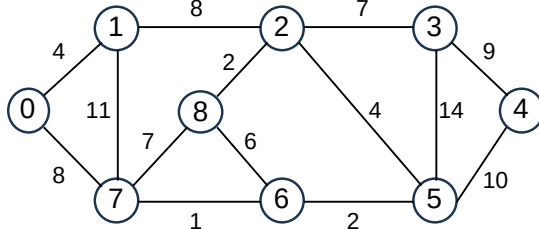
```
1   $n = W.rows$ 
2   $D^{(0)} = W$ 
3  for  $k = 1$  to  $n$ 
4      let  $D^{(k)} = (d_{ij}^{(k)})$  be a new  $n \times n$  matrix
5      for  $i = 1$  to  $n$ 
6          for  $j = 1$  to  $n$ 
7               $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$ 
8  return  $D^{(n)}$ 
```

Description: Input and Output Files (1/2)

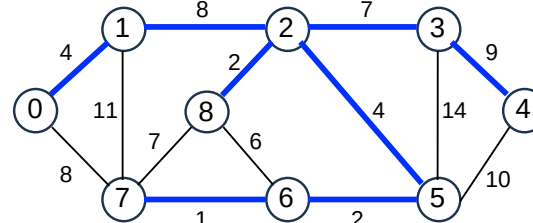
❑ Input file (**undirected** graph) ❑ Output file (Dijkstra's algorithm)

```
[num_vertices]<tab>[num_edges]<tab>[start vertex ID]
[vertex ID] <tab> [vertex ID] <tab> [weight]
...
[vertex ID] <tab> [vertex ID] <tab> [weight]
```

```
[vertex ID]<tab>[predecessor]
...
[vertex ID]<tab>[predecessor]
```



```
9 14 0
0 1 4
0 7 8
1 2 8
1 7 11
2 3 7
2 5 4
2 8 2
3 4 9
3 5 14
4 5 10
5 6 2
6 7 1
6 8 6
7 8 7
```

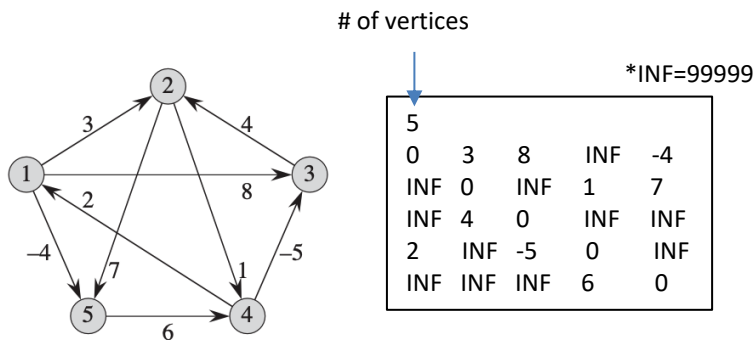


```
0 NIL
1 0
2 1
3 2
4 3
5 2
6 5
7 6
8 2
```

Description: Input and Output Files (2/2)

❑ Input file (**directed** graph)

❑ Output file (Floyd-Warshall)



$$w_{ij} = \begin{cases} 0 & \text{if } i = j, \\ \text{weight of } (i, j) & \text{if } i \neq j, (i, j) \in E, \\ \infty & \text{if } i \neq j, (i, j) \notin E. \end{cases}$$



D: shortest-path weight matrix

P: predecessor matrix

D	5	← # of vertices			
0	1	-3	2	-4	
3	0	-4	1	-1	
7	4	0	5	3	
2	-1	-5	0	-2	
8	5	1	6	0	
P	5	← # of vertices			
NIL	3	4	5	1	
4	NIL	4	2	1	
4	3	NIL	2	1	
4	3	4	NIL	1	
4	3	4	5	NIL	

*NIL=-1

$D[i][j] = \delta(i, j)$: the weight of a shortest-path from vertex i to vertex j

$P[i][j]$ = the predecessor of vertex j on a shortest-path from vertex i to vertex j

Makefile

□ Writing a makefile

```
all: compile

compile: graph.c
| gcc graph.c. -o graph

run: graph
| ./graph input_sssp.txt input_apsp.txt output_sssp.txt output_apsp.txt

clean: graph
| rm graph output_sssp.txt output_apsp.txt
```

Makefile

‘make’ will compile your code and generate the executable file
(e.g., gcc graph.c -o graph)

‘make run’ will run your program
(./graph input_sssp.txt input_apsp.txt output_sssp.txt output_apsp.txt)

4 filenames: 2 input files and 2 output files

‘make clean’ will delete the executable file and output file
(e.g., rm graph output_sssp.txt output_apsp.txt)

**Please make sure that your code is compiled and run successfully,
and the results in the output file are correct**

Q&A:
Assignment #3